

PENETRATION TESTING REPORT

HackThisSite — Basic Challenges (Levels 1–11)

Report Classification	CONFIDENTIAL — Internal Use Only
Target Application	HackThisSite Basic Challenge Series
Scope	Basic Levels 1 through 11
Assessment Type	Black-Box Web Application Penetration Test
Report Date	May 2026
Prepared By	Cyber Security Analyst — IT Security Team
Reviewed By	Chief Information Security Officer (CISO)

1. Executive Summary

This report presents the findings of a penetration test conducted against the HackThisSite Basic Challenge series (Levels 1–11). The assessment was performed in a legally sanctioned, controlled environment provided by HackThisSite for the purpose of cybersecurity skills development and awareness.

The assessment identified a range of vulnerabilities commonly found in real-world web applications, including missing server-side authentication, command injection, client-side bypass techniques, and server misconfiguration. These vulnerabilities, if present in a production banking environment, would represent severe risks to the confidentiality, integrity, and availability of organisational and customer data.

Of the 11 levels assessed:

- 2 vulnerabilities were rated Critical (OS Command Injection, SSI Injection)
- 4 vulnerabilities were rated High (Authentication Bypass, Data Exposure, Path Traversal, Insecure Cookies)
- 4 vulnerabilities were rated Medium (Backup File Disclosure, Weak Encryption, Email Redirect, Server Misconfiguration)

Immediate remediation is recommended for all Critical and High severity findings. The organisation should also review its secure development lifecycle to prevent similar vulnerabilities from being introduced into production systems.

2. Scope of Testing

Application Name	HackThisSite Basic Challenges
URL	https://www.hackthissite.org/missions/basic/
Levels in Scope	Basic Level 1 through Basic Level 11
Testing Method	Black-Box (no prior knowledge of source code)
Testing Period	May 2026
Tools Used	Browser Developer Tools, Page Source Review, Cookie Editor, Manual Injection
Legal Authority	HackThisSite Terms of Service — Legal, educational penetration testing environment

All testing was conducted strictly within the boundaries of HackThisSite's terms of service. No systems outside the designated challenge environment were targeted or accessed.

3. Risk Summary

Level	Title	Vulnerability Type	Severity
1	No Password Protection	Missing Authentication Control	High
2	Disabled JavaScript Validation Only	Client-Side Authentication Bypass	High
3	Password in HTML Source	Sensitive Data Exposure in Source Code	High
4	Backup File Disclosure	Insecure File Storage / Information Disclosure	Medium
5	Unencrypted Email-Based Password	Sensitive Data Transmitted Insecurely	Medium
6	Weak Encryption (Caesar Cipher)	Use of Weak / Reversible Encryption	Medium
7	Unix Command Injection	OS Command Injection	Critical
8	Server-Side Include (SSI) Injection	SSI Injection / Code Injection	Critical
9	Directory Traversal	Path Traversal / Insecure Direct Object Reference	High

10	Cookies and Weak Access Control	Insecure Cookie-Based Authentication	High
11	Apache .htaccess Misconfiguration	Server Misconfiguration / Information Disclosure	Medium

4. Vulnerability Findings

The following sections detail each vulnerability identified during the assessment, including reproduction steps, business impact, and recommended remediation.

Basic Level 1: No Password Protection			
Severity	High	Vulnerability Type	Missing Authentication Control
Description	The first level has no server-side password validation. The HTML source reveals the correct password in a comment or hidden field, or the form simply accepts any input.		
Steps to Reproduce	1. View page source (Ctrl+U). 2. Locate the password in an HTML comment or hidden input field. 3. Submit the discovered password to pass the level.		
Impact	Any unauthenticated user can bypass the login and gain full access by simply reading the client-side code.		
Recommendation	Never store or validate passwords on the client side. All authentication logic must reside server-side. Passwords must never appear in HTML source.		

Basic Level 2: Disabled JavaScript Validation Only			
Severity	High	Vulnerability Type	Client-Side Authentication Bypass
Description	Authentication relies exclusively on JavaScript running in the browser. Disabling JavaScript or modifying the script removes all access controls.		
Steps to Reproduce	1. Open browser Developer Tools. 2. Disable JavaScript or delete the validation function. 3. Submit the form — access is granted without any valid credential.		
Impact	Complete authentication bypass. Any attacker who disables JavaScript can gain unauthorised access instantly.		
Recommendation	Implement server-side authentication. Client-side validation is for user experience only and must never serve as a security control.		

Basic Level 3: Password in HTML Source			
Severity	High	Vulnerability Type	Sensitive Data Exposure in Source Code

Description	The password is embedded directly in the HTML page source as a hidden field or plaintext comment, visible to anyone who views the page source.
Steps to Reproduce	1. Right-click the page and select View Page Source. 2. Search for 'password', 'value', or 'hidden'. 3. Use the discovered value to authenticate.
Impact	Credentials are exposed to all users. No technical skill is required to exploit this vulnerability.
Recommendation	Remove all credentials from client-side code. Use server-side session management and hashed credential storage.

Basic Level 4: Backup File Disclosure

Severity	Medium	Vulnerability Type	Insecure File Storage / Information Disclosure
Description	A backup of the password file (e.g., password.php.bak) is stored in the web root with a predictable filename, allowing direct download.		
Steps to Reproduce	1. Guess common backup file extensions (.bak, .old, .txt). 2. Navigate to the backup URL (e.g., /password.php.bak). 3. Download the file and read the plaintext password.		
Impact	Sensitive configuration files containing credentials are publicly accessible, exposing backend logic and secrets.		
Recommendation	Never store backup files in the web root. Restrict access to sensitive directories. Implement proper file permission controls and automated scanning for exposed files.		

Basic Level 5: Unencrypted Email-Based Password

Severity	Medium	Vulnerability Type	Sensitive Data Transmitted Insecurely
Description	The application sends the password via an unencrypted email-based mechanism. By manipulating the form parameters, an attacker can redirect the password to their own email.		
Steps to Reproduce	1. Inspect the form action and hidden email field. 2. Modify the email field value to an attacker-controlled address. 3. Submit the form — the password is emailed to the attacker.		
Impact	Credentials can be intercepted or redirected to any attacker, resulting in full account compromise.		
Recommendation	Never transmit passwords via email fields controlled by user input. Use server-side generation of reset tokens with short expiry and enforce HTTPS.		

Basic Level 6: Weak Encryption (Caesar Cipher)

Severity	Medium	Vulnerability Type	Use of Weak / Reversible Encryption
-----------------	---------------	---------------------------	-------------------------------------

Description	The application encrypts the password using a Caesar cipher (character rotation). The encrypted value is visible in the source, and the cipher is trivially reversible.
Steps to Reproduce	1. View page source and locate the encoded password string. 2. Apply reverse Caesar cipher (ROT offset) to decode it. 3. Submit the plaintext password.
Impact	Security through obscurity provides no real protection. Any attacker with basic cryptographic knowledge can reverse the encoding instantly.
Recommendation	Use industry-standard hashing algorithms (bcrypt, Argon2) for password storage. Never use home-grown or weak encryption algorithms.

Basic Level 7: Unix Command Injection

Severity	Critical	Vulnerability Type	OS Command Injection
Description	User input is passed unsanitised directly to a Unix shell command. An attacker can inject arbitrary OS commands by appending shell metacharacters to the input.		
Steps to Reproduce	1. Identify input fields that interact with server-side commands. 2. Inject a shell metacharacter (e.g., ; ls, cat /etc/passwd). 3. Observe command output returned in the response.		
Impact	Full server compromise. An attacker can execute arbitrary commands, read sensitive files, escalate privileges, or establish persistence on the host.		
Recommendation	Never pass user input to OS commands. Use parameterised APIs instead. If shell execution is necessary, use strict input whitelisting and escape all metacharacters.		

Basic Level 8: Server-Side Include (SSI) Injection

Severity	Critical	Vulnerability Type	SSI Injection / Code Injection
Description	The application processes Server-Side Include directives embedded within user-supplied input. An attacker can inject SSI tags that execute commands on the server.		
Steps to Reproduce	1. Identify a field that reflects input on the server (e.g., a name field in a .shtml page). 2. Inject an SSI directive such as <!--#exec cmd='id'-->. 3. Observe command execution output in the server response.		
Impact	Remote code execution on the server. Attackers can read files, execute system commands, and potentially achieve full server takeover.		
Recommendation	Disable SSI processing unless strictly required. Sanitise and encode all user input before reflection. Apply Content Security Policy and output encoding.		

Basic Level 9: Directory Traversal

Severity	High	Vulnerability Type	Path Traversal / Insecure Direct Object Reference
-----------------	-------------	---------------------------	---

Description	The application includes files based on a user-supplied filename parameter without validation. An attacker can traverse the directory structure using ../ sequences to access arbitrary files.
Steps to Reproduce	1. Identify a URL parameter that references a file (e.g., ?file=page.html). 2. Replace the value with a traversal sequence (e.g., ../../etc/passwd). 3. Observe sensitive file contents returned in the response.
Impact	Unauthorised access to sensitive server files including configuration files, credentials, and system files.
Recommendation	Validate and canonicalise all file paths server-side. Use a whitelist of permitted filenames. Never expose raw file-system paths to users.

Basic Level 10: Cookies and Weak Access Control

Severity	High	Vulnerability Type	Insecure Cookie-Based Authentication
Description	The application grants administrator access based on a cookie value set client-side. By modifying the cookie, an attacker can impersonate an administrator without valid credentials.		
Steps to Reproduce	1. Inspect cookies using browser Developer Tools or a proxy. 2. Identify a cookie such as 'admin=0'. 3. Modify the value to 'admin=1'. 4. Refresh the page — administrator access is granted.		
Impact	Full privilege escalation. Any user can gain administrator-level access by trivially modifying a cookie.		
Recommendation	Never rely on client-controlled values for authorisation decisions. Use signed, server-side session tokens. Validate all access decisions server-side on every request.		

Basic Level 11: Apache .htaccess Misconfiguration

Severity	Medium	Vulnerability Type	Server Misconfiguration / Information Disclosure
Description	The web server is misconfigured such that .htaccess files or protected directories are accessible, leaking authentication configurations or allowing bypass of access restrictions.		
Steps to Reproduce	1. Attempt direct access to restricted directories or .htaccess files. 2. Probe for misconfigured Apache directives. 3. Use discovered information (e.g., AuthUserFile path) to locate and read the password file.		
Impact	Exposure of authentication configurations and potential bypass of directory-level access controls.		
Recommendation	Ensure Apache denies access to .htaccess files externally. Audit all directory configurations. Store credential files outside the web root and apply least-privilege file permissions.		

5. Key Recommendations

Based on the findings above, the following priority actions are recommended:

5.1 Immediate Actions (Critical & High)

- Migrate all authentication logic to the server side; remove any client-side validation used as a security control.
- Sanitise and validate all user input before passing it to OS commands, file systems, or include directives.
- Replace client-side cookie-based authorisation with signed, server-side session tokens.
- Never expose credentials or sensitive values in HTML source, hidden fields, or page comments.

5.2 Short-Term Actions (Medium)

- Implement a secure file management policy: remove backup files from the web root and audit for exposed sensitive files.
- Replace all weak or home-grown encryption with industry-standard algorithms (bcrypt, Argon2 for passwords; AES-256 for data).
- Disable SSI processing on web servers where it is not required.
- Enforce HTTPS across all application endpoints to prevent credential interception.

5.3 Long-Term / Strategic

- Adopt a Secure Software Development Lifecycle (SSDLC) that includes mandatory security reviews and threat modelling for all applications.
- Conduct regular penetration testing and vulnerability assessments against all public-facing systems.
- Implement a Web Application Firewall (WAF) to provide an additional layer of protection against injection attacks.
- Deliver regular security awareness training for all developers and IT staff, incorporating OWASP Top 10 and ACSC Essential Eight guidelines.
- Establish a vulnerability disclosure and patch management programme with defined SLAs for critical findings.

6. Conclusion

The HackThisSite Basic Challenge series demonstrates a broad spectrum of vulnerabilities that represent real and prevalent threats in web application security. The findings reinforce the critical importance of server-side validation, secure coding practices, and defence-in-depth strategies.

Organisations — particularly those in regulated sectors such as banking and finance — must treat web application security as a core operational risk. The vulnerabilities identified in this assessment map directly to common real-world breaches and underscore the need for continuous security testing, developer training, and robust security governance.

This report should be reviewed by the CISO and relevant IT stakeholders, and all findings should be tracked through to remediation with clearly assigned ownership and timelines.